Sky Cob: Design Document (Recoverable High-Altitude Balloon Payload)

Oct 14, 2026 · @2theMAX

Sky Cob is a small camera-and-radio package that a weather balloon carries high into the sky. On the way up it sends live video and its position to the ground; afterwards it comes down under a parachute so the team can find it and launch it again. UWOSLAB builds it from hobby drone electronics inside a 3D-printed shell shaped like an ear of corn.

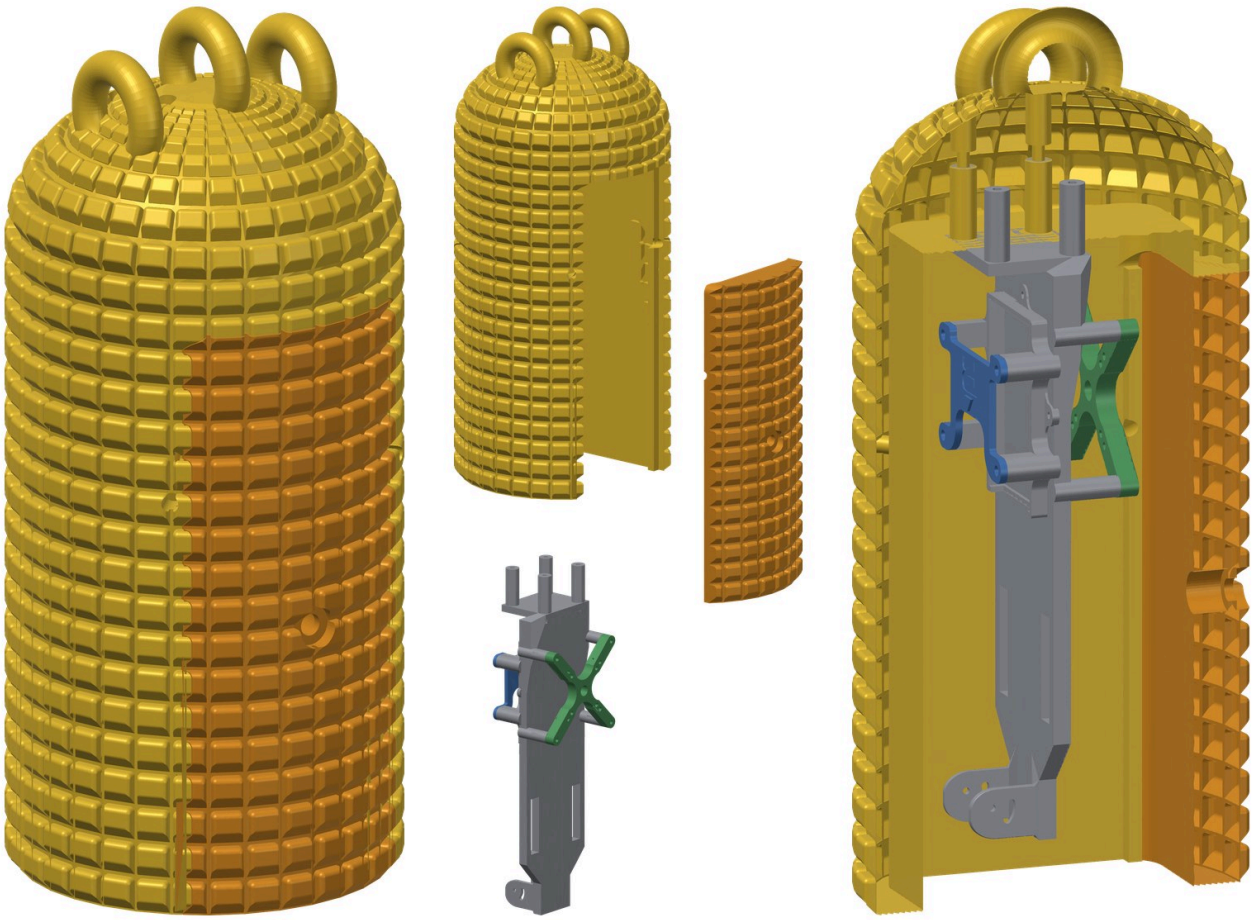
How a flight works

Sky Cob is a passenger, not an aircraft: nothing on board steers it. The balloon sets where it goes, and the electronics only watch, record and report.

1. **Before launch:** wait for a GPS lock (30 s to 3 min from a cold start, at least 5–8 satellites), then switch the payload to armed so it starts recording.
2. **Ascent:** the balloon lifts the cob by the parachute loops on its tip. A downward-facing camera streams live video to the ground, and the radio link sends back position, altitude, battery and attitude.
3. **Out of range:** the payload eventually flies beyond the radio's reach. It is set up to ignore that and keep recording, instead of trying to fly home or land the way a drone would.
4. **Descent:** once the balloon is spent, the cob comes back down under its parachute.
5. **Recovery:** a separate 5G GPS tracker reports where it landed, independent of the main electronics. The recorded flight log (the "blackbox") is read off its memory card afterwards.

A first look

The cob is about 113 mm across and about 250 mm tall to the top of its loops. All electronics hang on a single printed frame that slides up into it from below. These renders are from the current design files; none of these parts has been printed yet.



Left to right: the assembled cob from the back (orange = removable door); the parts pulled apart; a cutaway showing the electronics frame seated inside.

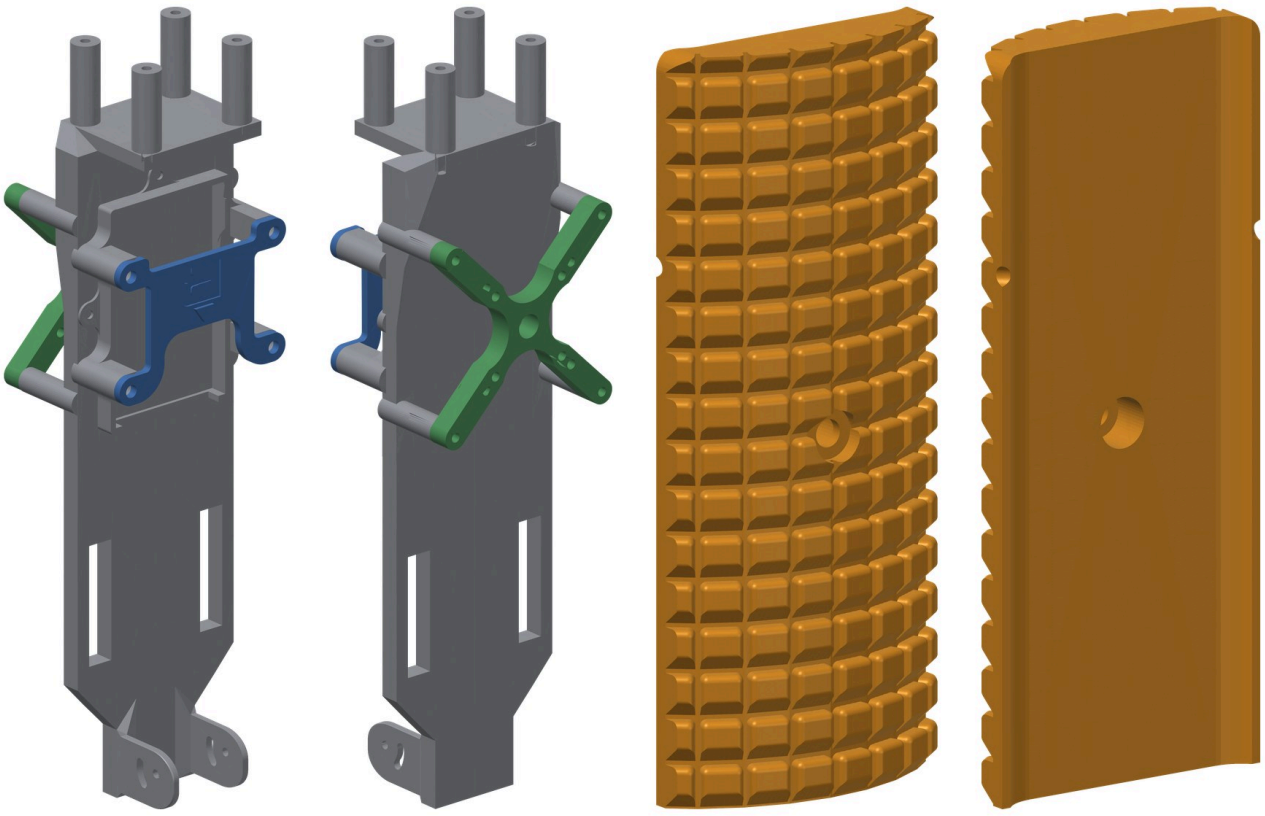
What's inside

The payload is built from parts made for hobby FPV (first-person-view) drones, which are small, light and made to send video and data over radio. Its "brain" is a flight controller running the open-source INAV firmware, used here only to read sensors, record and report.

Part	What it does	Model
Flight controller	Reads GPS and motion sensors, overlays flight data on the video, records the flight log	SpeedyBee F405 Wing, stock INAV 9.0.1

Part	What it does	Model
GPS	Position, altitude and speed	uBlox M10-class (HGLRC M100 Pro or Foxeer M10)
Radio receiver	Takes switch commands from the ground and sends data back	Radiomaster DBR4-TD, ExpressLRS
Camera	Points straight down at the ground	Analog FPV camera, 19 mm mount
Video transmitter	Streams the camera's picture to the ground	High-power analog VTX, 5917 MHz
Recovery tracker	Reports the landing spot independently of everything else	5G GPS tracker
Battery	Powers everything	3S (12 V) pack

The frame. One printed chassis carries everything. The flight controller sits under a cover plate (blue) on one face. The receiver and video transmitter share a cross-shaped plate (green) on the back. The camera hangs from a bracket at the bottom, and four posts on top lock the frame into the shell.



Left to right: the frame from the flight-controller side (blue plate); the frame from the back (green receiver/transmitter plate); the door from outside; the door from inside, with the antenna connector hole.

Talking to the ground

Three separate radio links connect the cob to the ground, so losing one doesn't lose the payload.

Link	Ground equipment	What travels over it
Control and telemetry (ExpressLRS)	Hobby RC transmitter running EdgeTX (Radiomaster Boxer preferred) with an external ELRS module (Radiomaster Nomad preferred); a laptop on the module's USB for tracking	Up: switch commands (arm, screen pages, beeper). Down: signal strength, GPS position, altitude, battery, attitude, mode
Live video	Any analog 5.8 GHz video receiver tuned to Raceband channel 8 (5917 MHz)	The downward camera view, with flight data drawn on top
Recovery	The tracker's own reporting, separate from the radio gear	Landing position

The transmitter's switches only arm the payload, flip between on-screen data pages and sound the beeper; none of them moves anything.

The corn-cob shell

The shell isn't drawn by hand: a script (`skycob_cob.py`) measures the electronics frame and builds the shell around it with a set gap on every side. If a part inside changes, re-running the script reshapes the shell to match.

Shape. The "kernels" are rounded square tiles on a 32-sided barrel. The underside of each tile is angled at 45°, so the cob 3D-prints standing upright without support material.

Mounting. The frame's top presses flat against the inside of the tip. Four posts on the frame plug into sockets there, and four M3 screws go down into them from outside through holes in the tip.

Openings. Four small vent holes sit around the middle, level with the radio plate. A removable door in the back gives access to the receiver and video transmitter, and the antenna connector (RP-SMA) passes through the door. The door currently just pushes in; nothing yet holds it shut.

Dimensions. Calculated from the design files, not measured from a print.

Feature	Value
Overall size	≈ 113 mm across the kernels; ≈ 230 mm body, ≈ 250 mm to the top of the loops
Electronics stack	≈ 51 × 44 × 166 mm
Inner cavity	≈ 71 × 79 × 186 mm, open at the bottom
Gap around the stack	10 mm sides and flight-controller face; 25 mm back; 20 mm at the bottom opening; 0 at the top
Wall	4 mm minimum, kernels 3.5 mm proud on top
Kernels	32 columns, 10.5 mm pitch, 1 mm grooves, 1.5 mm rounded edges
Tip	Rounded dome, 0.8 × the core radius long
Parachute loops	3 rings at −30 / 0 / +30 mm, 14 mm radius, 10 mm thick
Posts and sockets	Ø7 mm × 20 mm posts, 0.4 mm socket clearance, 5 mm plate above for the screw heads
Vents	4 × Ø5 mm, at 45° between the loops
Antenna port	Ø6.5 mm hole in a 2.5 mm panel, Ø12 mm flats for the nut, 40 mm below the vents
Door	Full cavity width, bottom to 5 mm above the radio plate; 4 × 4 mm rails; 0.3 mm undersize

Putting it together

Build everything onto the frame first, then slide the frame into the shell and fit the door last.

1. Flight controller into its bay, correctly oriented; cover plate on top with M2.5 screws, with no wiggle.
2. GPS on the top platform with VHB tape or glue. Which way it faces doesn't matter, because its compass isn't used.
3. Receiver and video transmitter bolted to the cross-shaped plate through its 20×20 mm holes with M2 screws. The plate goes on the rear 40×40 mm standoffs with the receiver facing in and the transmitter facing out.
4. 5G tracker on the rear of the frame between the two large holes, and the 3S battery on the other side.
5. Camera on the 19 mm bracket at the bottom, pointing straight down.
6. Slide the frame up into the cob until its posts seat in the sockets under the tip, then drive 4 M3 screws down through the holes in the tip.
7. Push the door onto its rails.

Where it stands

The project is a work in progress. The electronics setup is done and verified on the real flight controller, but the current shell (03 below) exists only as design files and hasn't been printed or test-fit.

01 — Hand-modeled shell

Files: `cobshell.stl`

The original shell, modeled by hand, standing upright.

Status: replaced by 03.

02 — Shell turned for a 250×250 mm print bed

Files: `fix model.stl`

01 rotated to fit a 250×250 mm printer. Same part, new orientation; the README's print list still points to it.

Status: replaced by 03.

03 — Generated corn cob (LATEST)

Files: `skycob_cob.step`, `skycob_door.step`, `cobby19mm_cam_stand_standoffs.step` — script `skycob_cob.py`

Built by script around the electronics frame, so it follows any change to the parts inside. New here:

- kernel tiles shaped to print upright without supports;
- frame posts that plug into sockets under the tip, held by screws from outside;
- 3 parachute loops on the tip;
- vents, an antenna port and a removable back door.

Outcome: not yet printed or test-fit.

Risks

The biggest risk right now is the beeper: the flight controller listens for it on a channel the transmitter never sends, and a beeper you can't trigger makes the cob harder to find on the ground.

Risk	Why it matters	Mitigation
Beeper can't be triggered	The flight controller expects the beeper on channel 9; the transmitter sends nothing there	Add a channel 9 switch on the transmitter, or move the beeper to a channel already in use
Untested fits	The post sockets (0.4 mm clearance), the zero-gap contact under the tip and the door rails (0.3 mm undersize) have never been printed	Print small test sections cut from the design files before the full cob
Too tall for the printer	The cob is an estimated ≈ 250 mm tall with its loops	Check in the slicer; shorten the tip or loops in the script and regenerate
Door falls out	It only pushes in, and must survive the jolt of the parachute opening	Add a screw, a snap tab or tape

Risk	Why it matters	Mitigation
Wrong sensor calibration	The saved settings include sensor offsets measured on the original flight controller	Recalibrate every new board
Calculated, not measured	Every dimension comes from the design files	Measure the first print and update the tables
Hard-to-edit frame file	The frame's STEP was converted from a triangle mesh, so CAD tools can't snap to its holes and faces	Use it for fit and printing; edit the original mesh and regenerate
Large shell file	`skycob_cob.step` is ≈ 46 MB, close to GitHub's 50 MB warning	Keep the 32-sided barrel; watch file size when adding detail
Radio rules	Video transmitter power and band must meet local regulations	Check the transmitter's manual and local rules before flight
Configurator error on start	The setup app throws an error at startup (from upstream code)	Harmless for setup; ignore

Design decisions

Topic	Decision
Losing the radio link	Do nothing: no return-home, no landing. A drone would try to come back; a balloon payload can't, so it stays armed and keeps recording
Firmware	Stock INAV 9.0.1, unmodified; only its saved settings change
Setting up a flight controller	Paste the saved settings file, or press one button in the custom setup app; sensor calibration is always done fresh
Mounting angle	Flight controller stands on edge (pitched 90°) and is allowed to arm at any angle
Shell design	Generated by script from the parts inside, not drawn by hand

Topic	Decision
Kernel shape	Square tiles on a 32-sided barrel, angled underneath: prints without supports and keeps the file small
Holding the frame	Four posts into sockets under the tip, screwed from outside; the frame top presses against the tip
Reaching the radios	Full-width removable door in the back wall
Finding it after landing	Separate 5G tracker that works even if the main electronics fail
Video channel	Raceband channel 8 (5917 MHz), full power

Technical reference

The details below are for whoever builds or configures a Sky Cob; a first-time reader can skip them.

Flight controller settings

Flash the stock `SPEEDYBEEF405WING` target with INAV 9.0.1, then load `INAV_9.0.1_cli_SKY_COB_20260701_003845.txt` in the INAV Configurator's CLI tab. It is a `diff all` batch (`batch start` → `defaults noreboot` → settings → `save`). The companion **sky-cob-configurator** app applies the same baseline with one "Apply Balloon Defaults & Reboot" button; on the real board it applied 98 settings with zero errors. Never reuse the dump's `acczero_*` , `accgain_*` and `gyro_zero_*` on another board: recalibrate in standard board orientation, not the mounted one.

Area	Setting
Safety (balloon-specific)	`failsafe_procedure = NONE` , `failsafe_min_distance_procedure = NONE` , `nav_rth_allow_landing = NEVER` , `nav_rth_climb_first = OFF` , `safehome_usage_mode = OFF`
GPS	`gps_sbas_mode = WAAS` ; Galileo, BeiDou and GLONASS on

Area	Setting
Orientation	`align_board_pitch = 900`, `align_mag = CW270FLIP`, `small_angle = 180`
Video	`vtx_band = 5` (Raceband), `vtx_channel = 8`, full power
Blackbox	Rate denominator 2; logs `NAV_POS NAV_PID MAG ACC ATTI RC_DATA RC_COMMAND MOTORS SERVOS`; microSD, only 4 GB usable
Identity / locale	`name = SKY COB`, `pilot_name = UWOSLAB`, `osd_units = IMPERIAL`, `tz_offset = -240`, `tz_automatic_dst = USA`

Channel map

As it actually is in `Sky Cob.yml` (`mixData`) and the CLI dump (`aux`). The README disagrees: it lists OSD Off and both OSD pages on CH6/SC, the beeper on CH7/SB, and arming on "Aux 5" (the FC uses AUX1 = CH5).

Channel	Radio source (`mixData`)	FC mode (`aux`)	Range (µs)
CH5 (AUX1)	SD	ARM	1300–2100
CH6 (AUX2)	SA	OSD OFF	1300–2100
CH7 (AUX3)	SB	OSD ALT 1	1300–1700
CH8 (AUX4)	SC	OSD ALT 2	1700–2100
CH9 (AUX5)	— (no mix)	BEEPER	1300–2100

Regenerating the shell

- Run `python skycob_cob.py` from the repo root (needs `pip install cadquery`; about 9 minutes). `--assembly` also writes `skycob_assembly.step` with the frame seated in the cob.
- The script reads `cobby19mm_cam_stand.stl`, `cobbyspeedy-top.stl` and `cobby20x40.stl` from its own folder and writes the STEP files beside itself.

- Every gap and hardware size is a constant at the top of the script (`CLEARANCE_*`, `MIN_WALL`, `DOME_RATIO`, `SOCKET_CLEAR`, `DOOR_GAP`, `LOOP_RAISE`, kernel and port constants). Reduce `DOME_RATIO` or `LOOP_RAISE` to make the cob shorter.
- The cob prints upright, tip up.

Output	Contents
`skycob_cob.step`	The shell, ≈ 46 MB
`skycob_door.step`	The removable door
`cobby19mm_cam_stand_standoffs.step`	The frame with its 4 posts and M2 pilot holes in the back pegs, ≈ 28 MB
`skycob_assembly.step`	With `--assembly` only; not committed

Glossary

Term	Meaning
Arm	Switch the flight controller into its active state; here it starts the flight log
Blackbox	The flight controller's onboard flight log, written to a microSD card
CadQuery	A Python library for writing 3D models as code; it generates the shell
EdgeTX	Open-source firmware for hobby RC transmitters; `Sky Cob.yml` is its model file
ELRS / ExpressLRS, CRSF	The long-range radio system and data protocol linking the transmitter and the payload
Failsafe, RTH	What a drone does when it loses the radio link, such as returning home (RTH); turned off here
Flight controller (FC)	The small computer board that reads the sensors and runs INAV

Term	Meaning
FPV	First-person view: flying or watching through a live onboard camera
INAV	Open-source firmware for flight controllers, normally used to fly planes and drones
OSD	On-screen display: flight data drawn over the live video
Raceband	A standard set of 5.8 GHz analog video channels; channel 8 is 5917 MHz
RP-SMA	A common screw-on antenna connector
STEP, STL	3D file formats: STEP holds exact CAD geometry, STL a triangle mesh for slicing
VTX	Video transmitter: sends the camera's picture to the ground